

GENERATIVE MUSIC TOOLS FOR CLOSED DAWs

Terry Ma
Carnegie Mellon University
terryma@cs.cmu.edu

Chris Donahue
Carnegie Mellon University
chrisdonahue@cmu.edu

ABSTRACT

Many of the most widely-used digital audio workstations (DAWs) provide only limited access to project state information. This limited access can hinder the integration of assistive AI tools. We introduce a read-only sidecar framework for bringing session-aware generative tools to closed DAWs. First, a DAW-specific adapter reads a saved project into a structured session representation. Next, a model adapter converts that state into conditioning information. Finally, AI suggestions are generated and validated outside the project before the user manually imports an accepted result. We instantiate this design for Logic Pro by parsing its undocumented binary `ProjectData` file, recovering note-level content and the organization of region instances into tracks. We use the recovered arrangement to condition the Anticipatory Music Transformer [1] and generate a new MIDI part to accompany or extend the existing material. Our demo demonstrates an end-to-end workflow from a saved Logic project to arrangement-conditioned MIDI that can be dragged back into the session.¹

1. INTRODUCTION

Most producers remain with the same digital audio workstation (DAW) for years because their projects, plug-ins, shortcuts, and creative habits accumulate around it. Thus, switching to a more programmable and open DAW is often not straightforward, even though it would provide better support for new tools. This creates a deployment problem for assistive music models. Many popular DAWs expose only limited project state information through their documented interfaces, making it difficult for an external tool to understand the music that a producer is currently writing.

Context-conditioned MIDI generation is one example where having project state information is necessary. A producer may sketch a melody and ask a model to extend it, fill a missing passage, or write another part alongside it. To do this, the model must receive the existing context (notes,

timing, etc.) from the DAW. When a DAW does not expose this information, the producer must manually export and assemble the context before generation, interrupting the workflow that the assistive tool is meant to support.

There are some existing tools that work for some open DAWs. Composer’s Assistant 2 performs controllable multitrack MIDI infilling inside REAPER through ReaScripts [2], while MIDI-GPT supports track- and bar-level infilling with musical attribute controls and describes an Ableton Live integration [3]. Hookpad Aria conditions melody and harmony suggestions on the current song by embedding a generative model in a purpose-built lead-sheet editor [4]. MIDInfinite demonstrates that a large-scale symbolic music model can run locally in a browser on commodity hardware [5].

In each case, the musical context is supplied through a scripting interface, a plug-in protocol, or an editor designed to expose it. Such context is inaccessible by default in many widely used commercial DAWs. Logic Pro is a prominent example. Although its Scripter environment processes MIDI events in real time, it provides no documented project-wide interface for enumerating tracks, regions, and their stored notes [6].

We introduce a read-only sidecar framework for bringing session-aware generative music tools to these closed DAWs that do not expose the required project state. We test this framework for Logic Pro by reading its saved project file directly. Our work contributes (i) a parser that recovers note-level content, track organization, and region identity from Logic’s undocumented binary `ProjectData` (ii) a model adapter that converts the recovered parts into conditioning controls for the Anticipatory Music Transformer (AMT) (iii) a companion application for inspecting the recovered session, generating and playing back a new MIDI part, and dragging an accepted result back into Logic Pro. The source project is never modified, providing an end-to-end path from a saved Logic project to MIDI conditioning.

2. LOGIC PARSING

Our goal is to recover the musical state required by a symbolic generation model without asking the user to export each track manually. We represent a Logic session as its project metadata (tempo, meter, and key) along with its tracks and region instances. Each region contains notes represented by onset, duration, pitch, and velocity on a shared 480-PPQ clock. Note onsets are relative to the beginning of their region rather than the beginning of the song, and repeated uses of a region are retained as separate

¹ Application and source code:

<https://github.com/terryTM/amt-logic-companion>
Video demo: <https://youtu.be/On28Ucru95s>



5. AI USAGE STATEMENT

AI coding assistants (Claude Code) were used to help implement parts of the pipeline, including building the parser and developing the app. All code was reviewed and tested by the author. In addition, AI was used to help refine some of the writing in the manuscript. All technical content and claims are the author’s own.

6. REFERENCES

- [1] J. Thickstun, D. Hall, C. Donahue, and P. Liang, “Anticipatory music transformer,” *arXiv preprint arXiv:2306.08620*, 2023.
- [2] M. E. Malandro, “Composer’s Assistant 2: Interactive Multi-Track MIDI Infilling with Fine-Grained User Control,” in *Proceedings of the 25th International Society for Music Information Retrieval Conference*, 2024, pp. 438–445.
- [3] P. Pasquier, J. Ens, N. Fradet, P. Triana, D. Rizzotti, J.-B. Rolland, and M. Safi, “MIDI-GPT: A controllable generative model for computer-assisted multitrack music composition,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 2, 2025, pp. 1474–1482.
- [4] C. Donahue, S.-L. Wu, Y. Kim, D. Carlton, R. Miyakawa, and J. Thickstun, “Hookpad aria: A copilot for songwriters,” in *Extended Abstracts for the Late-Breaking Demo Session of the 25th International Society for Music Information Retrieval Conference*, 2024.
- [5] X. Zhou, C. Ruan, Z. Zhao, T. Chen, and C. Donahue, “Local deployment of large-scale music ai models on commodity hardware,” *arXiv preprint arXiv:2411.09625*, 2024.
- [6] Apple Inc., “Scripter api overview in logic pro,” <https://support.apple.com/guide/logicpro/scripter-api-overview-lgce3905a48c/mac>, accessed September 9, 2026.